

Esame scritto

2 ore

2 parti:

1) Domande 2-3

2) Esercizi 2

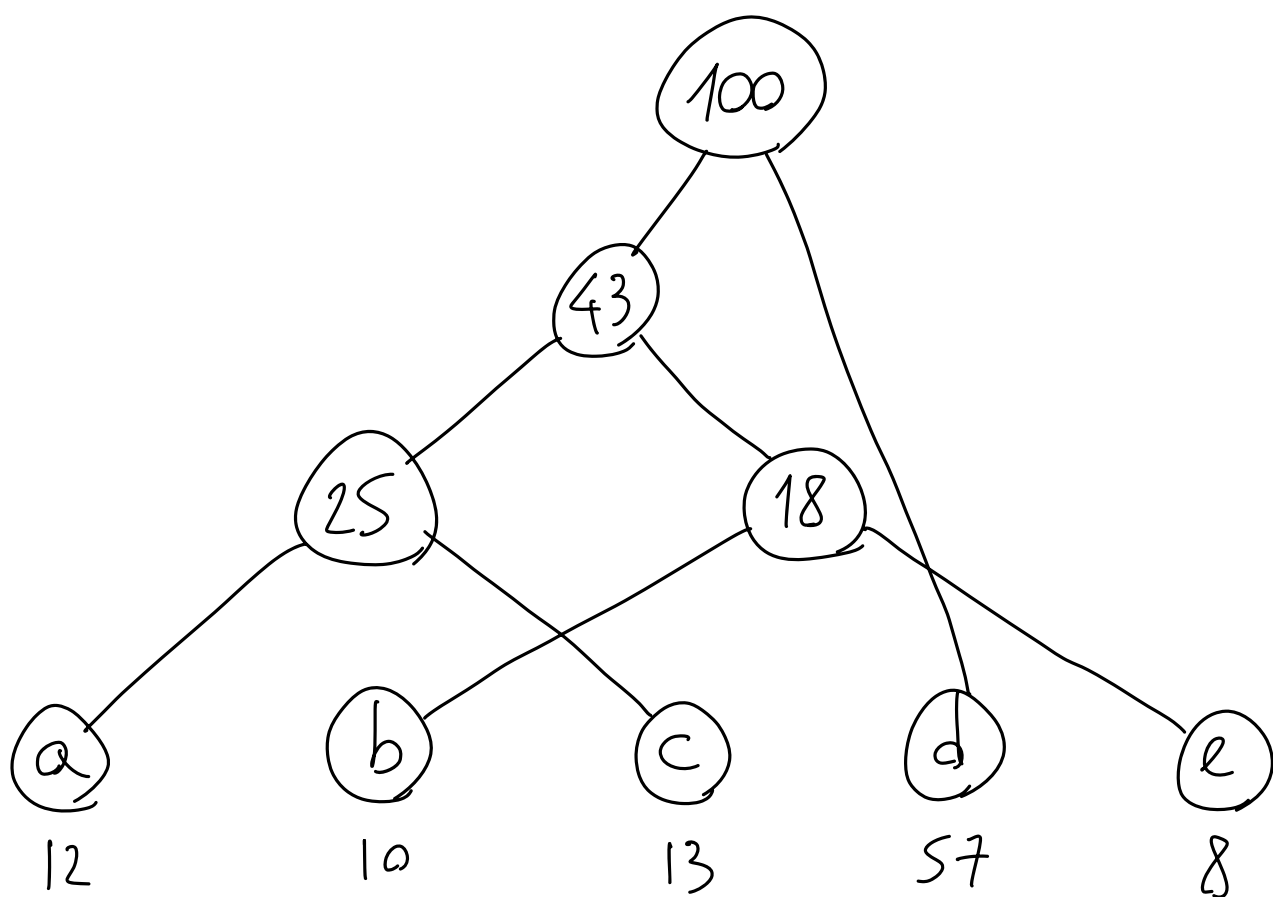
~32 punti in totale

Esercitazioni

Domanda (~5 punti): Indicare, in forma di albero binario, il codice libero da prefissi ottenuto tramite l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e\}$ supponendo che ogni carattere appaia con le seguenti frequenze:

a	b	c	d	e
12	10	13	57	8

Spiegare il processo di costruzione del codice

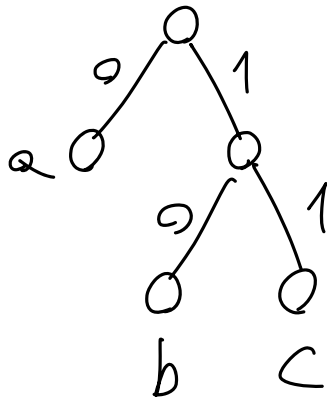


Esercizio (9 punti): Si consideri un file
 definito sull'alfabeto $\{a, b, c\}$ con frequenze
 $f(a)$, $f(b)$, $f(c)$. Per ognuna delle seguenti
 codifiche determinare, se esiste, un opportuno
 assegnamento di valori alle 3 frequenze per cui
 l'algoritmo di Huffman restituisce tale codifica,
 oppure argomentare che tale codifica non è mai ottenibile

- 1) $e(a) = 0$ $e(b) = 10$ $e(c) = 11$
- 2) $e(a) = 1$ $e(b) = 0$ $e(c) = 11$
- 3) $e(a) = 10$ $e(b) = 01$ $e(c) = 00$

2) $e(a)$ è prefisso di $e(c)$
 $\Rightarrow$ codice non libera da prefissi
 $\Rightarrow$ mai output di Huffman

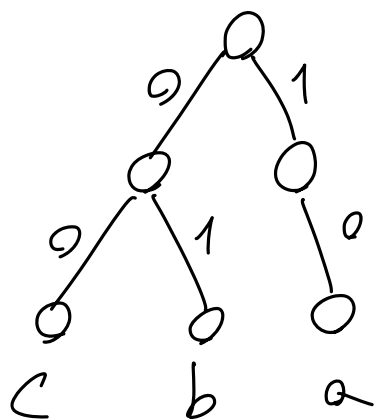
1)



$$f(b), f(c) < f(a)$$

per esempio: $f(a) = 50$ $f(b) = 25$ $f(c) = 25$

3)

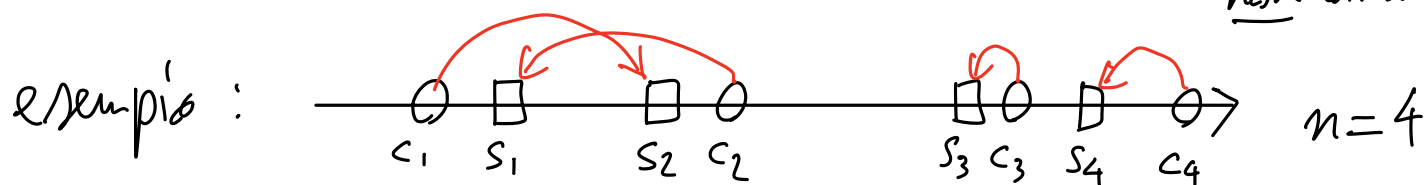


non è pieno
 $\Rightarrow$ non è ottimo
 $\Rightarrow$ mai output di Huffman

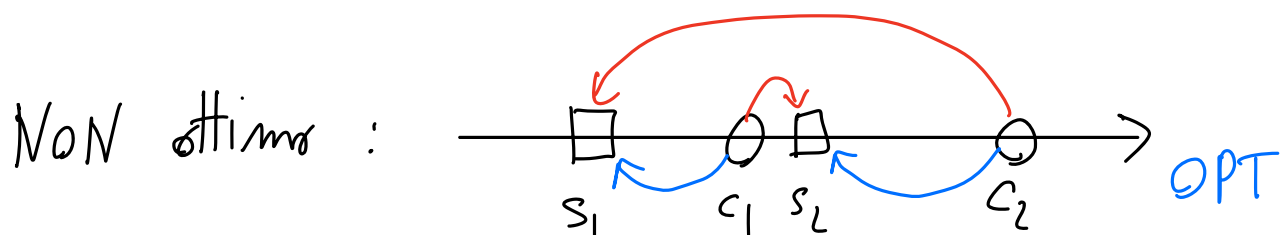
Esercizio: matching sulla linea

Sia $S = \{s_1, s_2, \dots, s_n\}$ un insieme di punti ordinati sulla linea reale, che rappresentano dei server. Sia $C = \{c_1, c_2, \dots, c_n\}$ un insieme di punti ordinati sulla linea reale, che rappresentano dei client. Il costo di assegnare un client c_i ad un server s_j è $|c_i - s_j|$. Fornire un algoritmo greedy che assegna ("match") ogni client ad un server distinto e che minimizzi il costo totale (equiv. medio) dell'assegnamento.

non ottimo



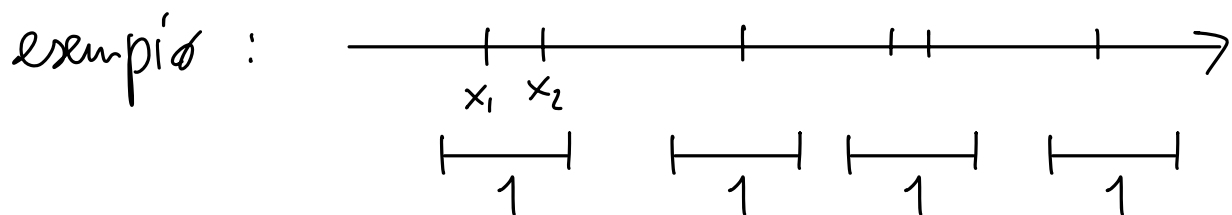
1) scegli C_1 al server + vicinato, poi $C_2, C_3, \dots$



2) $C_1 \rightarrow S_1, C_2 \rightarrow S_2, \dots, C_n \rightarrow S_n$

Per casa: dimostrare l'ottimalità di questo algoritmo

Esercizio: Sia $X = \{x_1, x_2, \dots, x_n\}$ un insieme di punti ordinati sulla linea reale. Fornire un algoritmo greedy che determini un insieme I di cardinalità minima di intervalli chiusi di ampiezza unitaria ($[a, b] \in I \Rightarrow b - a = 1$) tale che $\forall x_i \in X \exists j \in I$ tale che $x_i \in j$.



- 1) da s_x a d_x , inizia un nuovo intervallo sul primo punto non ancora coperto

MIN-COVER (X)

$n = \text{length}(X)$

$C = \{ [x_1, x_1 + 1] \}$

$\text{last} = 1$

for $i = 2$ to n do

if $x_i > x_{\text{last}} + 1$ then

$C = C \cup \{ [x_i, x_i + 1] \}$

$\text{last} = i$

return C

Per casa: dimostrare l'ottimalità di questo algoritmo